

FPGA RGB LED PWM Controller

Project report

Hardware Engineering SS26 - Team B2

Repository: [eggcitedraccoon/SS2026_HWE_Lab_B2](https://github.com/eggcitedraccoon/SS2026_HWE_Lab_B2)

1. Introduction and project scope

The project implements an RGB LED controller on an FPGA. The controller uses pulse-width modulation (PWM) to create different brightness levels on the red, green, and blue LED channels. By changing the duty cycle of these channels, the FPGA can produce solid colours, mixed colours, and animated lighting behaviour.

2. Requirements and responsibilities

2.1 Requirements

The project requirements keep the system small enough for an FPGA laboratory project while still requiring a complete chain from VHDL behaviour to visible output.

ID	Requirement
R1	The system shall be implemented on an FPGA.
R2	The system shall be described using VHDL.
R3	The system shall control an RGB LED.
R4	The system shall provide multiple selectable visual modes.
R5	The system shall allow the active mode to be selected using external switch inputs.
R6	The system shall produce visually aesthetic LED behaviour.

Figure 2. Requirement list used for the project.

2.2 Team responsibilities

Project task	Responsible student
Colour change	Petr Lavrenov
Fade module	Oleg Kelner
Blink module	Hany Chowdhury
Altium / PCB design	Hany Chowdhury
Top-level module	Oleg Kelner
PWM generator	Petr Lavrenov

3. System architecture

The design is modular. Each visible effect is implemented as a separate VHDL block, and the top-level module selects which effect drives the LED outputs.

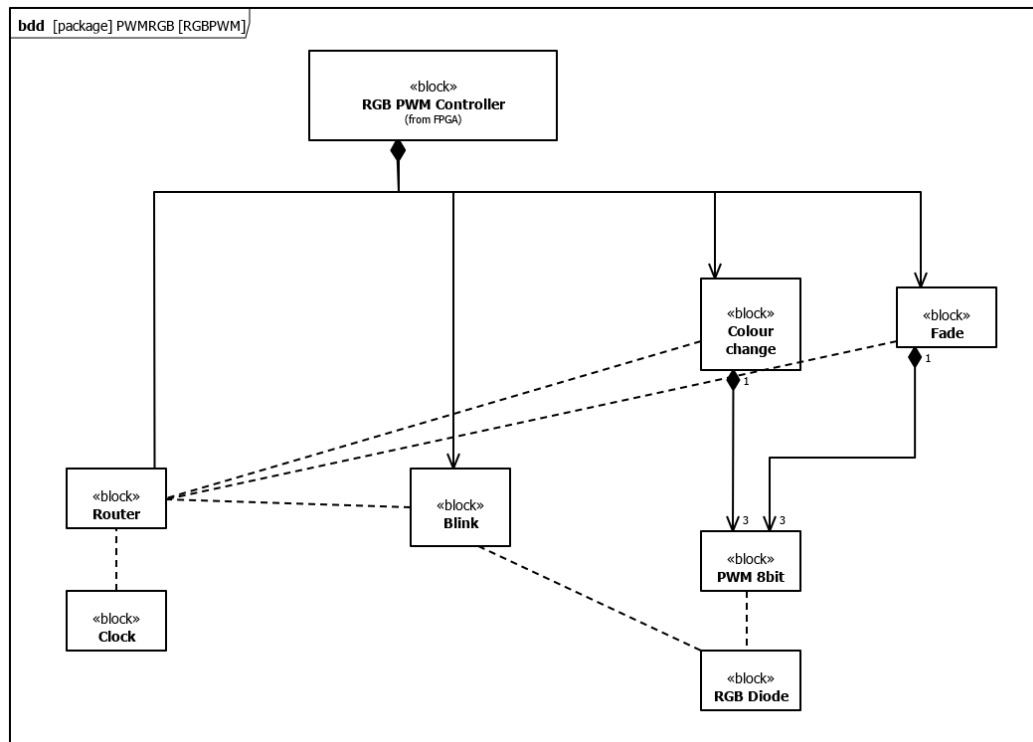


Figure 3. Block definition diagram for the RGB PWM controller.

A practical reading of the architecture is: switches select a mode; timing logic slows down the fast FPGA clock to human-visible changes; mode logic decides the colour values; PWM generators drive red, green, and blue outputs.

4. VHDL implementation

The main VHDL source folder contains the project modules `clock_router_top.vhd`, `clock_slowdown.vhd`, `colour_change.vhd`, `pwm_8bit.vhd`, `rgb_blink1.vhd`, `rgb_fade.vhd`, and `rgb_modes_top.vhd`. The top-level module instantiates the router and the three mode blocks, then routes the selected mode to LED16_R, LED16_G, and LED16_B.

Module	Main role
rgb_modes_top	Top-level integration block. Connects the mode router, the mode modules, and the final RGB outputs.
clock_router_top	Decodes switch vector A and enables the selected operating mode.
clock_slowdown	Generic tick generator for slow, visible timing.
pwm_8bit	8-bit PWM generator. Converts a brightness value into a duty cycle.
rgb_blink_top	Blink mode with repeated colour and off phases.
colour_change	Smooth colour-wheel transition using a hue counter.
rgb_fade_top	Fade in/fade out module using a brightness ramp and a colour list.

4.1 PWM generator principle

The PWM generator is the shared core mechanism. It increments an 8-bit counter and compares that counter with an 8-bit brightness input. A low brightness value makes the output high for only a small part of the PWM period, while a high brightness value keeps the output high for longer.

```
counter <= counter + 1;
if unsigned(brightness) = 255 then pwm_out <= 1;
elsif counter < unsigned(brightness) then pwm_out <= 1;
else pwm_out <= 0;
```

4.2 Mode selection

The lower two switch bits select the active mode. In the top-level module the output multiplexer maps the selected mode to the final RGB LED pins.

A(1 downto 0)	Selected behaviour
00	Blink mode
01	Smooth colour transition
10	Fade in / fade out
11	Outputs off / no active mode

4.3 VHDL libraries used

The VHDL files use the standard IEEE library declarations commonly used for synthesizable FPGA designs. The important packages for this project are:

- **ieee.std_logic_1164:** Provides std_logic, std_logic_vector, and rising_edge for FPGA signals.
- **ieee.numeric_std:** Provides unsigned arithmetic and conversions such as to_unsigned.

5. Operating modes

The implementation follows the three planned modes in the project README: colour blink, smooth colour transition, and fade in/fade out.

5.1 Colour blink mode

The blink module steps through a fixed sequence of colours and off phases. This is the simplest mode and works as a clear channel test: red, green, blue, and mixed colours can all be made visible.

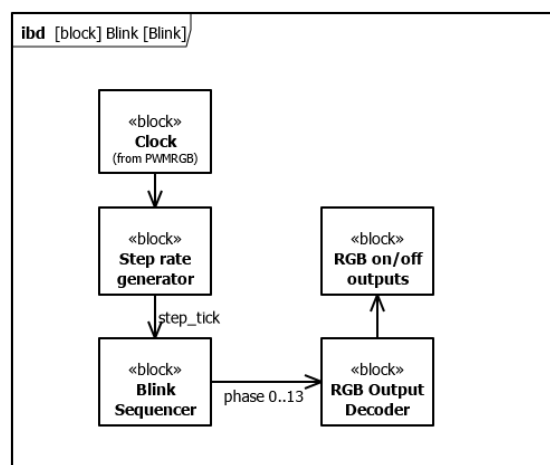


Figure 4. Blink mode diagram.

5.2 Smooth colour transition

The colour_change module uses a hue counter. The hue range is split into colour-wheel regions such as red to yellow (Hue = 0 to 255), yellow to green (256 to 511), green to cyan, cyan to blue, blue to magenta, and magenta to red (1280 to 1535). For every hue value, the module creates separate 8-bit red, green, and blue brightness values and feeds them into PWM generators.

Reason for the number 1535 is $1536/256=6$.

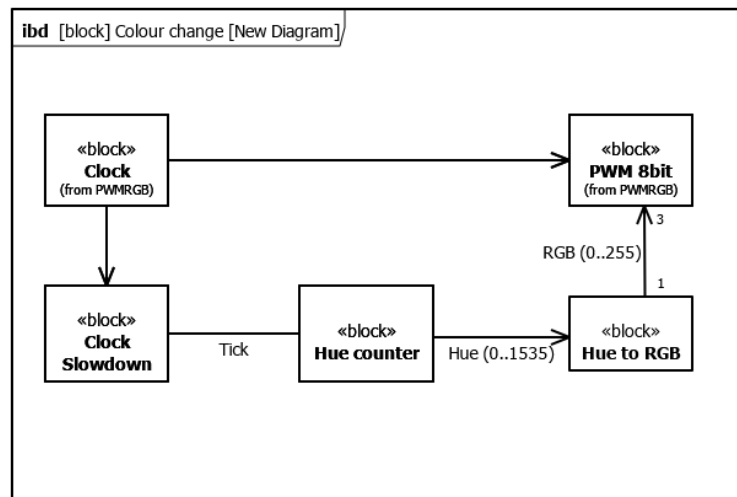


Figure 5. Smooth colour transition diagram.

5.3 Fade in / fade out mode

The fade module uses a shared brightness ramp. The active colour fades from black to full brightness and then back to black before the module moves to the next colour in the list. Because each colour channel is scaled by the same ramp, mixed RGB colours fade smoothly instead of simply switching on and off.

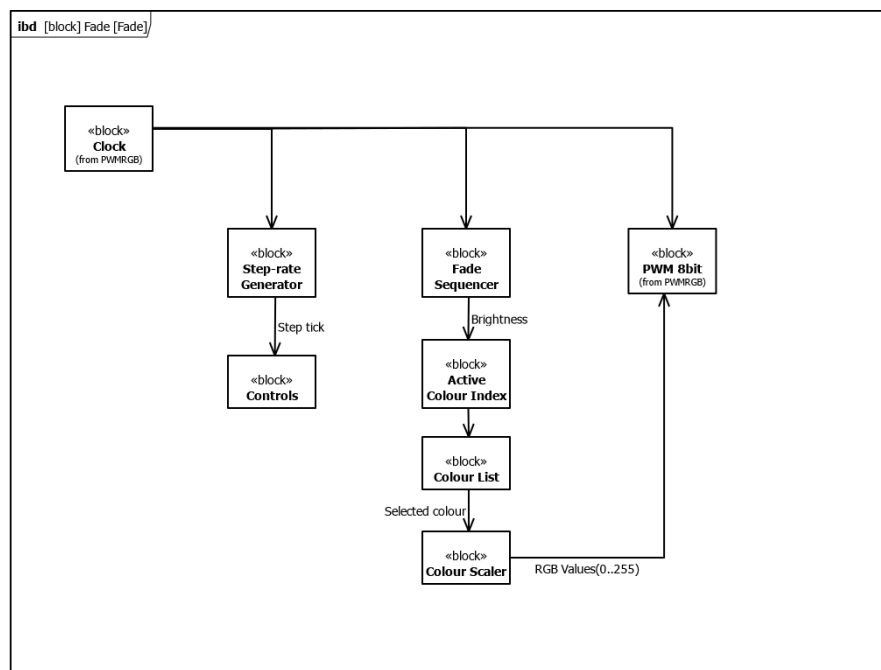


Figure 6. Fade mode diagram.

6. PCB / Altium documentation

6.1 Top-side PCB view

The top view places the main IC footprint near the centre. Around it are test points, the JTAG connector, switch footprints, decoupling capacitors, and the LED output area.

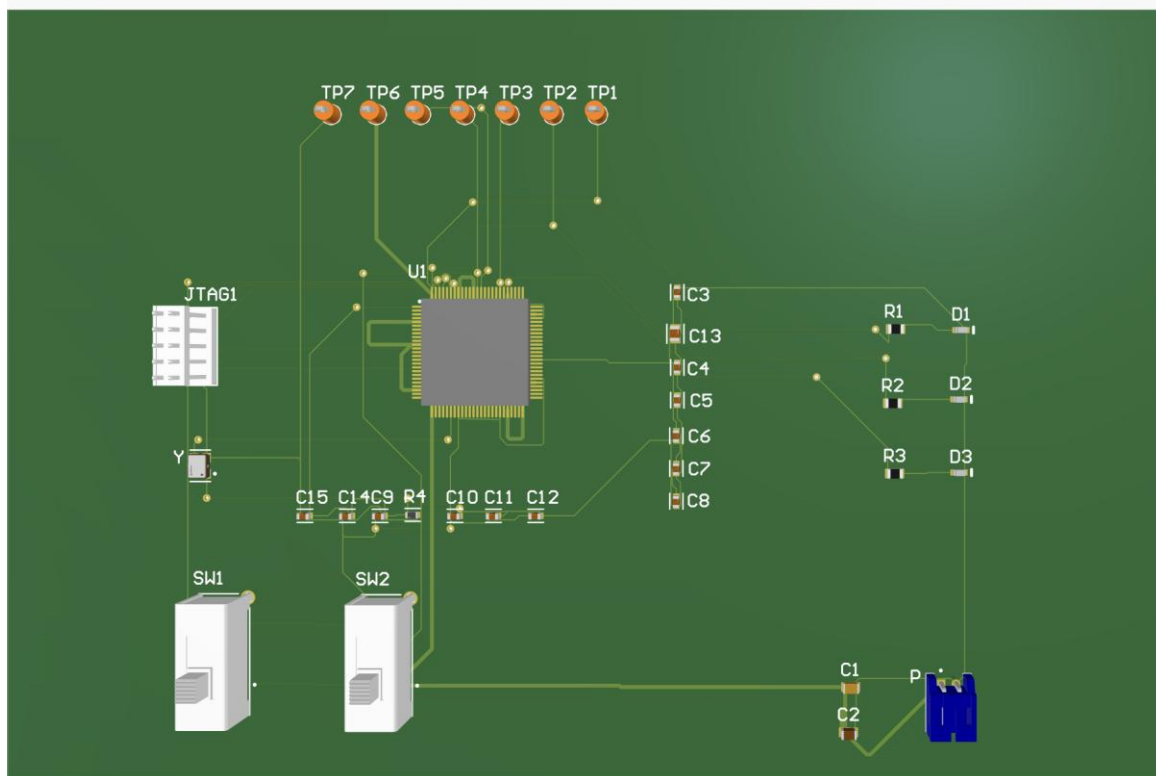


Figure 7. Altium PCB top view.

Part / Component	Example labels in PCB view	Function
FPGA / main controller IC	U1	Implements the VHDL logic of the RGB PWM controller. It generates the PWM signals, selects the active lighting mode, and drives the LED outputs.
RGB LEDs / LED outputs	D1, D2, D3	Provide the visible output of the system. Each LED channel is controlled by a PWM signal so that brightness and colour effects can be produced.
Current-limiting resistors	R1, R2, R3	Limit the current through the LED channels. Without these resistors, the LEDs or FPGA output pins could be overloaded.
Mode selection switches	SW1, SW2	Allow the user to select between different visual modes.
Test points / pin outputs	TP1–TP7	Provide access to important signals for testing and debugging.
Programming / debug connector	JTAG1	Allows the FPGA to be programmed and debugged.
Power connector	P	Supplies electrical power to the board. The controller, LEDs, and supporting components all depend on a stable supply voltage.
Decoupling capacitors	C3–C13, C15, etc.	Stabilise the supply voltage close to the IC pins. They help absorb short current peaks and reduce noise, especially when the FPGA switches many signals quickly.
Input/filter capacitors	C1, C2	Help smooth the incoming supply voltage and reduce disturbances from the

FPGA RGB LED PWM Controller - Project Report

<i>Part / Component</i>	<i>Example labels in PCB view</i>	<i>Function</i>
		<i>power input. They can also reduce voltage spikes during switching or connection.</i>
<i>Clock source / oscillator</i>	<i>Y</i>	<i>Provides a stable clock signal for the FPGA logic. The PWM counters and timing-based lighting effects depend on a regular clock.</i>
<i>Pull-up / pull-down or support resistor</i>	<i>R4</i>	<i>Can define a stable logic level or support an external signal, depending on the exact schematic connection. This avoids floating inputs and unreliable behaviour.</i>
<i>Pin headers / external connections</i>	<i>TP pins, connector pins</i>	<i>Allow external access to selected FPGA signals.</i>

7. Conclusion

The RGB PWM controller project demonstrates that even a relatively simple FPGA design can produce a clear and satisfying result. By combining VHDL-based PWM generation, selectable operating modes, external switch inputs, and RGB LED outputs, the system fulfils the main project goal: controlling an RGB LED in multiple visually distinct ways.

The project also showed that the “simple blinking light” is less trivial than it first appears. Behind the visible colour changes are timing logic, counters, output mapping, PCB components, power supply considerations, resistors, capacitors, test points, and programming access. In that sense, the project gave a practical look at how digital design becomes real hardware.

A slightly unexpected lesson was that cheap decorative LED products are probably cheap for a reason: once the control idea is understood, the basic principle is not very complicated. The system is simple enough to understand, but still produces an output that is immediately visible, easy to test, and quite nice to look at.